Flows II

Min-cost flow, Hopcroft–Karp, min-cut modeling, lower bounds

Learning goals

- Prove why **successive shortest paths** gives a min-cost flow
- Use **potentials** to run Dijkstra on residual graphs
- Implement **Hopcroft–Karp** and know König / path cover corollaries
- Model "choose a subset" problems as **min cuts**
- Handle **lower bounds**: feasibility, max flow, min flow

At this level, flow = modeling

Statement says...	Tool
"minimum total cost", "assign at min price"	min-cost max-flow
bipartite, 10^5 vertices; cover; independent set	Hopcroft–Karp
"choose a subset" with dependencies / penalties	min cut
"each edge carries at least l"	lower bounds

Graphs are usually small; Dinic / SSP run far below worst case.

Motivating problem — Task Assignment (CSES 2129)

n employees, n tasks, cost[i][j]. Each employee does exactly one task.
Minimize total cost; print the assignment.



Max flow = n; we want the **cheapest** max flow.

Successive shortest paths

Repeat: cheapest s–t path in the residual graph → push flow along it.
(Reverse residual edges have cost −w.)

Criterion: flow f is min-cost for its value
⇔ residual graph has **no negative cycle**.

SSP keeps the criterion at every step ⇒ every intermediate flow is optimal.

- Stop at value k → min-cost flow of value k
- Stop when path cost ≥ 0 → cheapest flow of any value

Potentials (Johnson)

Reduced cost: `w_π(u,v) = w(u,v) + π(u) − π(v)`
Path reduced cost = real cost + π(s) − π(t) ⇒ same shortest paths.

With π = shortest distances: all residual `w_π ≥ 0`,
path edges have `w_π = 0` ⇒ their reverses also 0.

After each Dijkstra: **π(v) += dist(v)** → stays non-negative.

Initial π: zeros if costs ≥ 0, else one Bellman–Ford (no negative cycles!).

MCMF — Dijkstra step

```
bool dijkstra(int s, int t) {
    fill(dist.begin(), dist.end(), INF); dist[s] = 0;
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (int id : g[u]) {
            const E& ed = e[id];
            ll nd = d + ed.cost + pot[u] - pot[ed.to];   // >= 0
            if (ed.cap > 0 && nd < dist[ed.to]) {
                dist[ed.to] = nd; pe[ed.to] = id; pq.push({nd, ed.to});
            }
        }
    }
    if (dist[t] == INF) return false;
    for (int v = 0; v < n; v++) if (dist[v] < INF) pot[v] += dist[v];
    return true;
}
```

MCMF — augmentation

```
pair<ll,ll> run(int s, int t, ll maxFlow = LLONG_MAX) {
    initPotentials(s);                          // SPFA if negative costs
    ll flow = 0, cost = 0;
    while (flow < maxFlow && dijkstra(s, t)) {
        ll f = maxFlow - flow;
        for (int v = t; v != s; v = e[pe[v] ^ 1].to) f = min(f, e[pe[v]].cap);
        for (int v = t; v != s; v = e[pe[v] ^ 1].to) {
            e[pe[v]].cap -= f; e[pe[v] ^ 1].cap += f;
        }
        flow += f;
        cost += f * (pot[t] - pot[s]);      // real path cost
    }
    return {flow, cost};
}
```

O(F · m log n) + one Bellman–Ford.

MCMF modeling patterns

- **Assignment:** bipartite, unit caps, cost on middle edges
- **k disjoint shortest paths:** unit caps, cost = length, flow k (CSES 2130)
- **Convex costs:** parallel unit edges with marginal costs 1, 3, 5, ... (convexity ⇒ cheap copies used first) — CF 863F
- **Degree limits:** split node into "parent side" / "child side" — CF 277E
- **Max profit:** negate costs — beware negative cycles

Hopcroft–Karp: phases

1. **BFS** from all free left vertices → layers by alternating distance
2. **DFS** along layer+1 edges only → maximal set of disjoint shortest augmenting paths
3. Dead ends: `dist[u] = -1` → each phase O(E)

```
bool dfs(int u) {
    for (int v : adj[u]) {
        int w = matchR[v];
        if (w == -1 || (dist[w] == dist[u] + 1 && dfs(w))) {
            matchL[u] = v; matchR[v] = u; return true;
        }
    }
    dist[u] = -1; return false;
}
```

Why O(E√V)

- Shortest augmenting path length **increases** every phase
- After √V phases: every augmenting path has > √V vertices
- $M \oplus M^*$ has $|M^*| - |M|$ **disjoint** augmenting paths
  ⇒ $(|M^*| - |M|) \cdot \sqrt{V} < V$ ⇒ fewer than √V left
- Each later phase adds ≥ 1 ⇒ ≤ √V more phases

2√V phases × O(E) = O(E√V).

What a matching buys you

Bipartite fact	Formula
König	min vertex cover = max matching
Independent set	max IS = V − max matching
DAG path cover (disjoint)	n − matching on split graph
Hall	left-saturating $\Leftrightarrow$ $\lvert N(S)\rvert \geq \lvert S\rvert$ $\forall S$

Cover from final BFS: Z = alternating-reachable from free left; **cover = (L \ Z) ∪ (R ∩ Z).**

Min cut as a choice

Vertex on S side = "chosen". Each penalty = an edge cut exactly when it applies.

Penalty	Edge
a if v **not** chosen ($v \in T$)	s → v, cap a
b if v chosen ($v \in S$)	v → t, cap b
c if $u \in S$, $v \in T$	u → v, cap c
forbid u chosen without v	u → v, cap ∞

Min cut = min total penalty.

Project selection (max-weight closure)

Projects (profit p), tools (cost c), project needs tools.

```
s ──p_i──▶ project_i ──∞──▶ tool_j ──c_j──▶ t
```

Answer = Σ p – min cut. Source side of min cut = optimal choice.

CF 1082G *Petya and Graph*: edges = projects (profit w),
vertices = tools (cost a_v).

Petya and Graph — code

```
int S = n + m, T = n + m + 1;
Dinic d(n + m + 2);
for (int v = 0; v < n; v++) { ll a; cin >> a; d.addEdge(v, T, a); }
ll total = 0;
for (int i = 0; i < m; i++) {
    int u, v; ll w; cin >> u >> v >> w; u--; v--;
    total += w;
    d.addEdge(S, n + i, w);        // lose w if edge i not taken
    d.addEdge(n + i, u, INF);      // taking edge i forces u, v
    d.addEdge(n + i, v, INF);
}
cout << total - d.maxflow(S, T) << "\n";
```

Min-cut practicalities

- **Vertex capacity:** split v → v_in → v_out
- **Recover the cut:** S = reachable from s in residual (CSES 1695)
- **∞ =** `LLONG_MAX / 4` — never `LLONG_MAX` (overflow)
- "Different sides" rewards: only if those pairs form a **bipartite** graph (flip one side); otherwise it is max-cut — NP-hard

Lower bounds: circulation

Edge (u, v) with l ≤ f ≤ c:

1. Edge capacity → c − l (optional part)
2. `excess[v] += l`, `excess[u] -= l`
3. S → v (excess > 0), v → T (excess < 0)
4. Feasible ⟺ max flow S→T = Σ positive excess

Flow on e = l + flow on the reduced edge.

Lower bounds: s–t flow

- Add **t → s, cap ∞** → s–t flow becomes a circulation
- Feasible value = flow on t → s
- **Max flow:** delete t → s, then + maxflow(s, t) in residual
- **Min flow:** delete t → s, then – maxflow(t, s) in residual
- **Min cost:** same with MCMF; forced units cost Σ l·w up front

Lower bounds — code

```
for (auto [u, v, lo, hi] : edges) {
    d.addEdge(u, v, hi - lo);
    excess[v] += lo; excess[u] -= lo;
}
ll need = 0;
for (int v = 0; v < n; v++) {
    if (excess[v] > 0) { d.addEdge(S, v, excess[v]); need += excess[v]; }
    if (excess[v] < 0) d.addEdge(v, T, -excess[v]);
}
int ts = d.addEdge(t, s, INF);
if (d.maxflow(S, T) != need) return -1;      // infeasible
ll flow = d.e[ts ^ 1].cap;                   // feasible value
d.e[ts].cap = d.e[ts ^ 1].cap = 0;           // delete t -> s
return flow + d.maxflow(s, t);
```

Complexity summary

Algorithm	Bound	Notes
Dinic	$O(V^2E)$	$O(E\sqrt{V})$ unit bipartite
Hopcroft–Karp	$O(E\sqrt{V})$	small constant
SSP + potentials	$O(F \cdot E \log V)$	+ one Bellman–Ford
Lower bounds	2 × max-flow	same algorithm inside

Common mistakes

- Negative cycles in MCMF (negated profits on cyclic graphs)
- No initial potentials with negative costs
- Adding INF distances of unreachable vertices to π
- 32-bit total cost
- Min-cut edge in the wrong direction — draw the 2-vertex case
- Lower bounds: wrong feasibility total; t → s edge not deleted
- Non-bipartite "different sides" penalties — not a min cut

Practice problems

- Review: CSES 1694, 1695, 1711
- Matching: CSES 1696, SPOJ MATCHING, CF 1404E
- MCMF: CSES 2129, CSES 2130, Kattis mincostmaxflow, CF 863F, CF 277E
- Min cut: CF 1082G
- Lower bounds: LOJ 115 / 116 / 117, CF 1288F

Summary

- MCMF = SSP; optimal ⇔ no negative residual cycle; potentials ⇒ Dijkstra
- Hopcroft–Karp: O(√V) phases; König, independent set, path cover
- Min cut: penalties as edges cut exactly when they apply; ∞ for implications
- Lower bounds: subtract l, balance excess with S/T, t → s ∞ edge
- Next: **Lecture 6 — Advanced strings** (suffix array + LCP, suffix automaton)